



Political Patricians (patricians)

Ein römischer Zensor überwacht N Patrizier mit fragwürdigen politischen Loyalitäten. Jeder Patrizier gehört **genau einer** geheimen politischen Fraktion an und jede Fraktion hat mindestens einen Patrizier. Der Zensor möchte ermitteln, welche Patrizier welcher Fraktion angehören.

Um dies zu erreichen, wird der Zensor wiederholt Bankette organisieren, zu denen alle der N Patrizier eingeladen sind. Bei jedem Bankett können die Patrizier an verschiedenen Tischen platziert werden, wobei der Zensor die Sitzordnung bestimmt. Ein Patrizier ist *einsam*, wenn kein anderer Patrizier der selben Fraktion an seinem Tisch sitzt.

Als Zeremonienmeister wird der Zensor nach jedem Bankett wissen, welche Patrizier einsam waren. Deine Aufgabe ist es, dem Zensor dabei zu helfen, die politischen Fraktionen der Patrizier zu bestimmen, ohne allzu viele Bankette zu organisieren.

Implementierung

Du musst eine einzelne .cpp Quelldatei einreichen.



Unter den Anhängen zu dieser Aufgabe findest du eine Vorlage `patricians.cpp` mit einer Beispielimplementierung.

Du musst die folgende Funktion implementieren:

C++	<code>vector<int> find_factions(int N)</code>
-----	---

- Die Ganzzahl N stellt die Anzahl der Patrizier dar. Die Patrizier sind von 0 bis $N - 1$ durchnummeriert.
- Die Funktion soll ein Array P aus N Ganzzahlen zurückgeben, welches die politischen Fraktion der Patrizier beschreibt. $P[i]$ sollte gleich $P[j]$ sein genau dann, wenn Patrizier i und j der gleichen politischen Fraktion angehören. Es gibt keine weiteren Einschränkungen für die Werte der Ganzzahlen in P .

Dein Programm kann die folgende Funktion aufrufen, welche bereits im Grader definiert ist:

C++	<code>vector<bool> organize_banquet(const vector<int>& Q)</code>
-----	--

- Das Array Q aus N Ganzzahlen stellt die Sitzordnung dar: Patrizier i wird an Tisch $Q[i]$ platziert. Es gibt keine Einschränkungen für die Werte der Ganzzahlen in Q : Tische können beliebig nummeriert werden und die einzige relevante Information ist, dass wenn $Q[i] = Q[j]$, die Patrizier i und j am gleichen Tisch platziert werden.
- Die Funktion wird ein Array A aus N Booleans zurückgeben. $A[i]$ ist wahr genau dann, wenn Patrizier i beim Bankett einsam war.

Der Grader ist **nicht adaptiv** und jede Einsendung wird anhand der selben Testfälle bewertet. Das heisst, die politischen Fraktionen jedes Patriziers sind in jedem Testfall im Voraus festgelegt und hängen nicht von den Aufrufen von `organize_banquet` durch Dein Programm ab.

Beispielgrader

Das Verzeichnis der Aufgabe enthält eine vereinfachte Version des Jury-Graders, die du verwenden kannst, um deine Lösung lokal zu testen. Der vereinfachte Grader liest die Eingangsdaten aus der Datei `stdin`, ruft die Funktion `find_factions` auf und schreibt schliesslich die Ausgabe in die Datei `stdout` mit folgendem Format.

Die Eingabe besteht aus 2 Zeilen, die Folgendes enthalten:

- Zeile 1: die Ganzzahl N .
- Zeile 2: N Ganzzahlen, welche die politische Fraktion jedes Patriziers beschreiben.

Die Ausgabe besteht aus einer einzigen Zeile, welche die Punktzahl enthält, die deine Lösung im Testfall erzielt hat. Wenn die Punktzahl 0 beträgt, wird eine kurze Nachricht in `stderr` mit dem Grund gedruckt (falsche Fraktionen als Antwort gegeben, ungültige Abfrage oder zu viele Abfragen).

Einschränkungen

- $4 \leq N \leq 2000$.

Du darfst `organize_banquet` höchstens 30000 mal aufrufen.

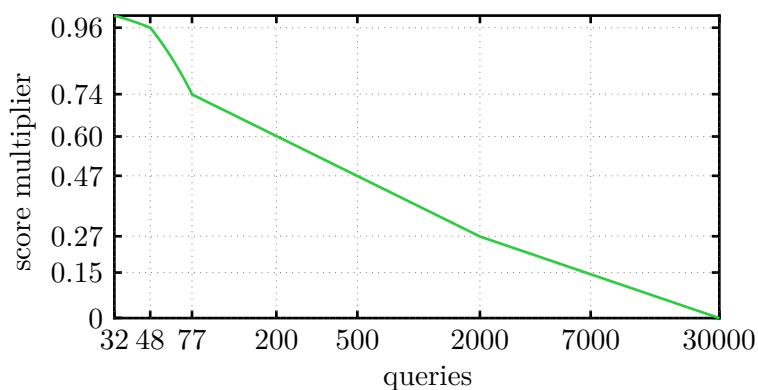
Punktevergabe

Dein Programm wird anhand einer Reihe von Testfällen getestet, die nach Teilaufgaben gruppiert sind. Der einer Teilaufgabe zugeordnete Punktemultiplikator ist das Minimum der in jedem Testfall erzielten Punktemultiplikatoren.

- **Teilaufgabe 0 [0 Punkte]**: Beispiel-Testfälle.
- **Teilaufgabe 1 [19 Punkte]**: Es gibt genau zwei politische Fraktionen.
- **Teilaufgabe 2 [29 Punkte]**: Jede politische Fraktion hat genau zwei Patrizier.
- **Teilaufgabe 3 [52 Punkte]**: Keine zusätzlichen Einschränkungen.

Der in einem Testfall erhaltene Punktemultiplikator wird wie folgt ermittelt.

Anzahl der Abfragen	Punktemultiplikator
≤ 32	1.0
$33 \leq x < 48$	$1.08 - 0.0025x$
48	0.96
$48 < x < 77$	$1.32413 - 0.00758x$
77	0.74
$77 < x < 2000$	$1.366 - 0.1 \log_2(x)$
2000	0.27
$2000 < x \leq 30000$	$1.027 - 0.0691 \log_2(x)$



Die für eine Teilaufgabe erzielte Punktzahl ist der minimale Punktmultiplikator, der über ihre Testfälle erzielt wurde, multipliziert mit der Basispunktzahl der Teilaufgabe.

Beispiele

Grader	Lösung
<code>find_factions(5)</code>	
	<code>organize_banquet([0,0,1,1,2])</code>
<code>return [1,1,1,1,1]</code>	
	<code>organize_banquet([0,0,0,1,1])</code>
<code>return [0,1,0,1,1]</code>	
	<code>organize_banquet([0,1,2,1,1])</code>
<code>return [1,0,1,0,1]</code>	
	<code>return [0,1,0,1,2]</code>

Grader	Lösung
<code>find_factions(4)</code>	
	<code>organize_banquet([1,1,0,0])</code>
<code>return [1,1,1,1]</code>	
	<code>organize_banquet([1,1,1,2])</code>
<code>return [0,1,0,1]</code>	
	<code>organize_banquet([0,1,1,1])</code>
<code>return [1,0,1,0]</code>	
	<code>return [0,1,0,1]</code>

Erklärung

In the first example, patricians 0 and 2 are in one political faction, patricians 1 and 3 are in a different one, and patrician 4 is in its own faction. In the first banquet, all patricians are lonely. In the second banquet, patricians 0 and 2 are not lonely because they are seated together. In the third banquet, patricians 1 and 3 are not lonely because they are seated together.

Im ersten Beispiel gehören die Patrizier 0 und 2 einer politischen Fraktion an, die Patrizier 1 und 3 einer anderen, und Patrizier 4 gehört seiner eigenen Fraktion an. Beim ersten Bankett sind alle Patrizier einsam. Beim zweiten Bankett sind die Patrizier 0 und 2 nicht einsam, weil sie zusammensitzen. Beim dritten Bankett sind die Patrizier 1 und 3 nicht einsam, weil sie zusammensitzen.